Bagh-Chal Game-Playing Agent: Minimax and Alpha-Beta with a Designed Evaluation Function

BCSE306L Artificial Intelligence · Module 3 (Adversarial Search) · VIT Vellore Pranav Shankar (24BCE2773) · Vishal (24BCE2802) · Shalom Santosh (24BCE2731)

> **Authorship.** Claude Code wrote this report's text on 2026-10-22 at Pranav's direction (PRD §6.1, §10); see the declaration in §8. Sections follow PRD §9 (course-mandated). Items marked **[CHECK]** are facts only the team can confirm.

---

1. Problem statement and motivation

Bagh-Chal ("move the tiger") is a traditional two-player game from Nepal. It is played on the 25 points of a 5×5 alquerque board, where diagonal lines pass only through every other point. One player has four tigers, which start on the corners. The other has twenty goats, which start off the board and are placed one per turn. Tigers capture by jumping over a goat, as in draughts, and win after five captures. Goats cannot capture; they win by surrounding the tigers until none of them can move.

The game is a good test bed for adversarial search for three reasons.

- **It fits the textbook model exactly.** It is deterministic, fully observable, turn-based and zero-sum, so minimax applies without modification.
- **The two sides are asymmetric.** They have different pieces, moves and goals. A symmetric evaluation such as "material difference" doesn't exist, so the evaluation function has to be designed.
- **The branching factor changes with the phase.** While goats are being placed, a position has about 21 moves; once all goats are on the board, it has about 8. One program therefore lets us see how pruning behaves at two very different branching factors.

The game tree is also too large to solve by brute force in Python, yet small enough that a depth-4 search answers in milliseconds. That makes the measurements practical.

We answer two questions (PRD §1):

- **(a)** How many nodes does alpha-beta pruning save over plain minimax, and how does the saving change with depth and phase?
- **(b)** Does our six-feature evaluation function play better than a naive one that only counts captured goats?
- **(c)** *(added for the final review)* What are alpha-beta's best case and worst case, how close does our agent come to each, and when is each algorithm the better choice?

In short: at depth 4, alpha-beta expands 14% of minimax's nodes (8.4% in the placement phase) and returns the same root score in all 90 searches. The designed evaluation beats the baseline in 96% of games as tiger, but in only 32% as goat. Move order alone moves alpha-beta between about 86% of minimax's nodes (worst order) and 2.5% (best order), and a cheap heuristic order reaches 4.8% (E3).

2. AI formulation

We model Bagh-Chal as a two-player, deterministic, fully observable, zero-sum game, using the formulation of Russell & Norvig (Ch. 5): an initial state, the legal actions, a transition model, a terminal test and a utility

function. Tigers are MAX and goats are MIN. From most positions, a depth-limited search cannot reach the end of the game. At the depth limit, a cutoff test therefore replaces the terminal test, and a heuristic evaluation estimates the utility.

Bagh-Chal has regional variants (for example, whether captures are compulsory and how repetition is handled). We fixed one set of rules, R1–R11, before writing any code and never changed it, so every experiment measures the same game.

2.1 Rules (frozen, PRD §4)

#	Rule
R1	Points `(r, c)`, `0 ≤ r, c ≤ 4`, index `5r + c`. Adjacent if they differ by ≤ 1 in each coordinate; diagonal links only through points with `r + c` even. 56 edges; degrees 3 (×12), 4 (×4), 5 (×4), 8 (×5).
R2	Jump lemma: if `y ∈ N(x)` and `z = 2y − x` is on the board, then `z ∈ N(y)` on the same line. 80 directed jumps.
R3	Start: tigers on the four corners, 20 goats in hand, 0 captured, goats move first.
R4	Turns alternate goat, tiger, goat, …
R5	While goats are in hand (`h > 0`), the goat's only move is to PLACE on an empty point.
R6	Once `h = 0`, a goat STEPs to an adjacent empty point. Goats never jump.
R7	A tiger STEPs to an adjacent empty point, or JUMPs over an adjacent goat to the empty point beyond. The goat is captured (`k += 1`). One jump per move; capturing is optional.
R8	Terminal, in order: `k ≥ 5` → tigers win; tiger to move with no move → goats win; goat to move with no move → tigers win; position repeated 3 times → draw; 200 plies → draw.
R9	Tigers are MAX. Utility `+1,000,000 − d` (tiger win), `−1,000,000 + d` (goat win), `0` (draw), with `d` = plies from the search root.
R10	Fixed move order: placements by ascending point; goat steps by (from, to); tiger moves by (from, to), steps and jumps interleaved.
R11	Ties keep the first best move (strict `>` for MAX, `<` for MIN).

2.2 Formal problem definition

Item	Definition
State	`s = ⟨B, h, k, p, H, n⟩`: board B over 25 points, goats in hand h, goats captured k, player to move p, repetition history H, ply count n
Initial state	R3
Actions	`PLACE(x)` while h > 0; `STEP(x, y)` to an adjacent empty point; `JUMP(x, y, z)` for tigers (R5–R7)
Transition model	`RESULT(s, a)`: apply the move, flip p, update h or k, append to H, increment n
Terminal test	R8
Utility	R9
Cutoff	`CUTOFF(s, d) = TERMINAL(s) ∨ d = 0`
Evaluation	§3.3: one scalar from the tiger's side
Branching factor	~21 in goat placement, ~10–20 in goat movement, ~8–16 for tigers; measured effective b* in Table 2
Complexity	Minimax O(b^d); alpha-beta best case O(b^(d/2))

One evaluation keeps the game zero-sum. The two sides make progress in different ways: tigers by capturing and threatening, goats by blocking and trapping. A natural idea is to give each side its own score.

Then, however, one side's gain would no longer be exactly the other side's loss. The game would become general-sum, and the argument that lets alpha-beta skip moves would no longer hold.

We instead combine both kinds of progress into **one number from the tiger's side**: tiger features are added and goat features are subtracted (§3.3). Tigers maximise this number and goats minimise it, so every point gained by one side is lost by the other. `alpha` and `beta` are then bounds on the same quantity, and pruning is sound. The evaluation is also bounded: `|EVAL| ≤ 896`, which is far below the smallest win or loss value, `1,000,000 − 200`. A position that is merely good can therefore never outrank a real win.

2.3 AI and ML in this project

The agent is a **rational agent built on classical AI**. Its environment is two-player, deterministic, fully observable and zero-sum, and its decisions come from **adversarial search**: minimax and alpha-beta, depth-limited, with a cutoff. A **hand-designed heuristic evaluation** (§3.3) scores positions at the cutoff. The six features and their weights encode our knowledge of the game.

The project uses **no machine learning**. There is no training data, no learned parameter and no training loop. The weights were set by reasoning (§3.3), the agent plays identically every time, and E2's games only measure it; they never adjust it. This was deliberate: the course module is adversarial search, and PRD §2 and C1 exclude learned weights and external AI services from the agent.

ML could be added later by learning the weights through self-play or TD-learning, or by replacing `EVAL` with a learned network. The search would stay the same. Separately, an AI *tool*, Claude Code, was used to write much of the code and this report; that is declared in §8. Full explanation: `docs/ai_ml_usage.md`.

3. Algorithm description

3.1 Minimax

Minimax assumes that both players play optimally. MAX picks the child with the highest value, MIN picks the lowest, and the value of a position is the result of that alternation down to the depth limit. Our version returns the value together with the **principal variation (PV)**: the line of best moves for both sides.

```
function MINIMAX(s, depth, ply) returns (value, pv)
    nodes ← nodes + 1                          # count every call, first
    if TERMINAL(s) then return (UTILITY(s, ply), [])
    if depth = 0   then return (EVAL(s), [])
    best ← −∞ if MAX to move else +∞;  pv ← []
    for each a in ACTIONS(s), in the fixed R10 order do
        (v, line) ← MINIMAX(RESULT(s, a), depth − 1, ply + 1)
        if (MAX to move and v > best) or (MIN to move and v < best) then
            best ← v;  pv ← [a] + line
    return (best, pv)
```

Three details matter for the experiments.

- **The node is counted first.** Every call counts itself, including the root and calls that stop immediately at a terminal or cutoff. Minimax and alpha-beta are counted the same way, which makes E1 a fair comparison. A depth-1 search expands exactly `1 + (number of legal moves)` nodes, and we test this.
- **Terminal comes before cutoff.** If a game is over at the depth limit, its exact result is used rather than the heuristic's guess.
- **`ply` is passed to the utility.** A win is worth `1,000,000 − ply`, so a win found nearer the root scores higher. The agent prefers the fastest win and the slowest loss, instead of wandering between equally

winning moves.

`depth` counts down (plies left to search), while `ply` counts up (distance from the root). With branching factor `b` and depth `d`, minimax takes `O(b^d)` time and `O(b·d)` memory.

3.2 Alpha-beta pruning

Alpha-beta computes the same value as minimax but skips subtrees that cannot affect it. It carries two bounds down the tree:

- `alpha` is the best value MAX can already guarantee on the path to the current node;
- `beta` is the best (lowest) value MIN can already guarantee.

Once `alpha ≥ beta`, the player above will never allow play to reach this node, so its remaining children are skipped.

```
function ALPHABETA(s, depth, ply, α, β) returns (value, pv)
    nodes ← nodes + 1
    if TERMINAL(s) then return (UTILITY(s, ply), [])
    if depth = 0   then return (EVAL(s), [])
    if MAX to move then
        best ← −∞
        for each a in ACTIONS(s) do
            (v, line) ← ALPHABETA(RESULT(s, a), depth − 1, ply + 1, α, β)
            if v > best then best ← v;  pv ← [a] + line
            α ← max(α, best)
            if α ≥ β then break               # MIN above will never allow this node
    else                                      # MIN to move: mirror image
        best ← +∞
        for each a in ACTIONS(s) do
            (v, line) ← ALPHABETA(RESULT(s, a), depth − 1, ply + 1, α, β)
            if v < best then best ← v;  pv ← [a] + line
            β ← min(β, best)
            if α ≥ β then break
    return (best, pv)                         # root call: α = −∞, β = +∞
```

Hand trace (Russell & Norvig Fig. 5.2). The root A is MAX. Its three MIN children are B, C and D, with leaves 3 12 8 | 2 4 6 | 14 5 2.

Step	Node	What happens	α, β at A
1	B	leaves 3, 12, 8 → B = 3	α = 3
2	C	first leaf 2 → C ≤ 2. Since 2 ≤ α = 3, MAX will never choose C: **cut**, so leaves 4 and 6 are never generated	α = 3
3	D	leaves 14, 5, 2 → D = 2	α = 3
4	A	max(3, ≤2, 2) = **3**	—

Minimax expands 13 nodes (1 + 3 + 9). Alpha-beta expands 11, because two leaves are skipped. The value is 3 either way: C's value can only be 2 or lower whatever the skipped leaves hold, and MAX already has 3. Both numbers are unit tests.

How much alpha-beta saves depends on move order. With the best move always first, it expands about `O(b^(d/2))` nodes; with the worst order, it expands as many as minimax. We use the fixed rule order R10 with no reordering, so E1 measures plain alpha-beta.

3.3 Evaluation function

`EVAL(s) = w1·capt + w2·mob + w3·threat − w4·trapped − w5·conn − w6·hub` (tiger's side)

Feature	Favours	Definition	Weight
`capt`	Tigers	goats captured, k	100
`mob`	Tigers	legal tiger STEP moves, summed over tigers	3
`threat`	Tigers	distinct goats some tiger could jump right now	15
`trapped`	Goats	tigers with no legal STEP or JUMP	40
`conn`	Goats	board edges with goats on both ends	2
`hub`	Goats	sum of degree over goats on even points	1

Bound: `|EVAL| ≤ 896` (tiger side) and `≤ 344` (goat side), far inside ±100,000, so no heuristic value outranks a real win (±1,000,000). Baseline for E2: `capt` only.

Why these features and weights. The weights were chosen by reasoning about the game, not tuned; the initial values were kept.

- **`capt` (100)** has the largest weight. A capture is permanent and five end the game, so one capture must outweigh any realistic change in the positional terms.
- **`trapped` (40)** comes next. Trapping all four tigers wins for the goats, but a trapped tiger can be freed later, so it counts for less than a capture.
- **`threat` (15)** is about a seventh of a capture. A goat that can be jumped right now is often lost, but the goats may still defend it. Each goat is counted once, however many tigers attack it.
- **`mob` (3)** is small. Each tiger step is one more way to keep threatening, but mobility ranges up to about 30, so a small weight per step is enough.
- **`conn` (2)** rewards goats standing next to each other. Such goats block landing squares and protect one another. Each link is worth little, but there are up to 56.
- **`hub` (1)** rewards goats on the high-degree even points, which cut the tigers' diagonal lines. It has the smallest weight because it overlaps with `conn`: goats on those points tend to be connected anyway.

The three tiger features measure what tigers want (captures, threats, room to move) and the three goat features measure what goats want (trapped tigers, a solid wall, control of the diagonals). Every feature takes about 100 table lookups to compute, so evaluation costs about 2 µs per call.

4. Implementation notes

The full implementation notes are Appendix B (source: `docs/implementation_notes.md`). They cover the state representation, topology generated from R1, the jump lemma, move generation and repetition, node counting, tie-breaking, evaluation computation and position generation. Paragraphs there tagged **[DRAFT]** were written by Claude Code for an owner to check; each owner deletes the tag once they have reviewed or rewritten their paragraph.

The program is plain Python 3 with no third-party libraries in the engine. A position is an immutable tuple, so a search can hold many positions without one move corrupting another. All board tables are generated at start-up from rule R1 rather than typed in by hand. The search sees the game only through five functions, so the same code is tested on the textbook tree and then run on Bagh-Chal.

Table 0. Verification

Check	Result
Unit and acceptance tests (`python -m pytest -q`)	301 passed
Topology (R1/R2)	56 edges, 13 even points, degree histogram {3:12, 4:4, 5:4, 8:5}, 80 jumps
Perft from the start, depths 1–4	21 · 252 · 5,052 · 68,204
Textbook tree (R&N Fig. 5.2)	value 3; minimax 13 nodes; alpha-beta 11 nodes
Engine contract (`python -m experiments check-engine`)	0 failures
Move ordering (E3, `tests/test_ordering.py`)	best order = Knuth–Moore leaf count and worst-case tree = b^d, exactly, for b ∈ {2,3,4}, d ∈ {2..5}; minimax count order-independent
Rules speed (`benchmarks/bench_rules.py`, Python 3.15, Apple silicon)	~1.04 M `legal_moves`/s; ~1.29 M `apply`/s
Evaluation speed	~1.8 µs per `designed_eval` call

5. Experiments and results

5.1 E1: Minimax vs alpha-beta

Setup: 30 fixed positions (15 placement, 15 movement, generated with seed 2026; see §4), depths 2, 3 and 4, designed evaluation. Nodes per search; time is the median of 3 runs.

Table 1. Median nodes and time per move

Phase	Depth	Minimax nodes	Alpha-beta nodes	AB / MM	Minimax ms	Alpha-beta ms
Placement	2	240	79	37.8%	0.664	0.211
Placement	3	3,055	671	26.2%	8.296	1.954
Placement	4	45,746	2,627	8.4%	117.7	6.939
Movement	2	68	46	66.7%	0.273	0.154
Movement	3	615	252	47.3%	1.890	0.836
Movement	4	4,536	818	18.6%	17.8	2.715
All	2	93	56	53.6%	0.346	0.192
All	3	922	332	35.4%	3.041	0.988
All	4	6,996	1,214	14.1%	24.7	3.887

Table 2. Effective branching factor b* (solves `N = 1 + b* + … + b*^d`)

Phase	Depth	b* minimax	b* alpha-beta
Placement	2 / 3 / 4	14.97 / 14.16 / 14.36	8.35 / 8.39 / 6.88
Movement	2 / 3 / 4	7.70 / 8.14 / 7.93	6.23 / 5.94 / 5.06

Correctness: alpha-beta returned the same root score as minimax on all 90 position/depth pairs, and the same move on all 90 (Appendix Table A3).

Figure 1. `results/e1_nodes_vs_depth.png`: nodes expanded per move against depth, log scale, placement and movement side by side.

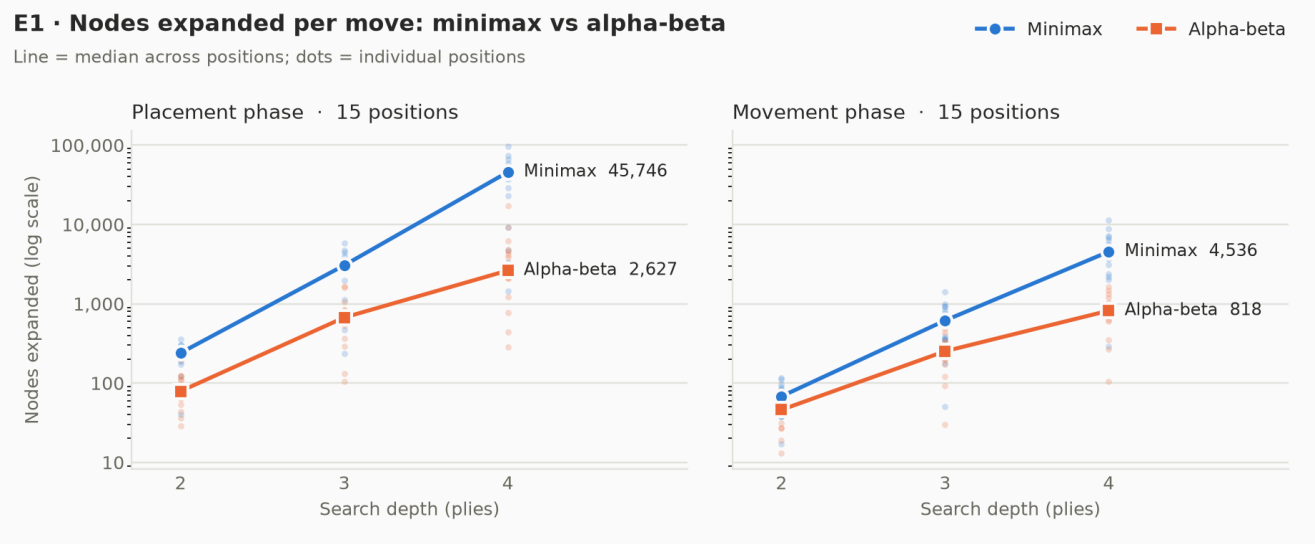
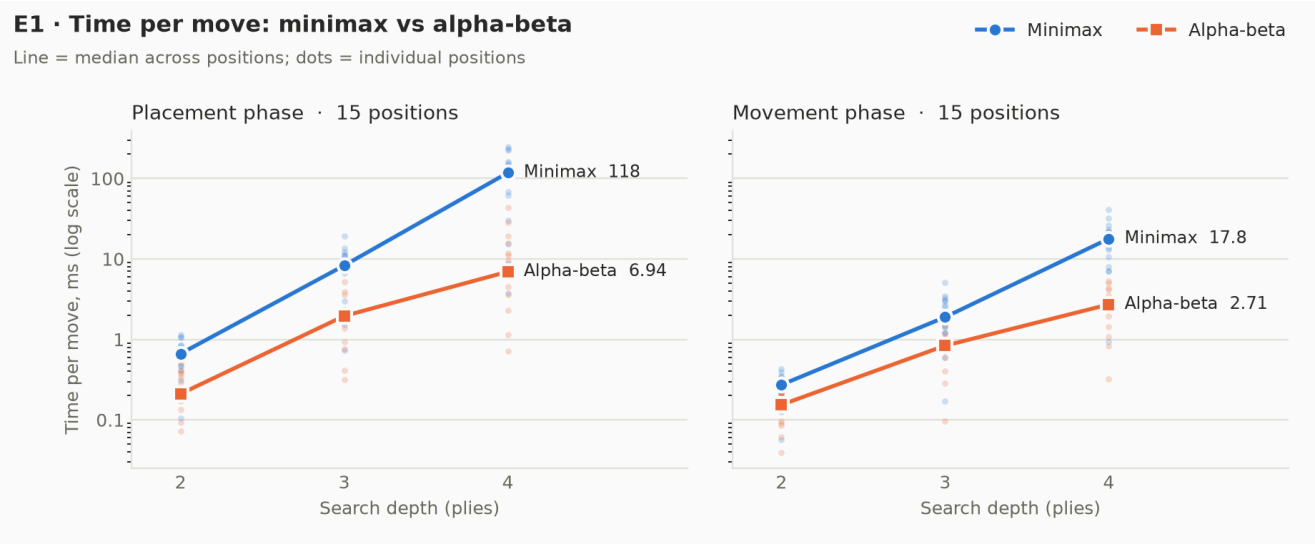


Figure 2. `results/e1_time_vs_depth.png`: time per move, same layout.



5.2 E2: Designed evaluation vs baseline

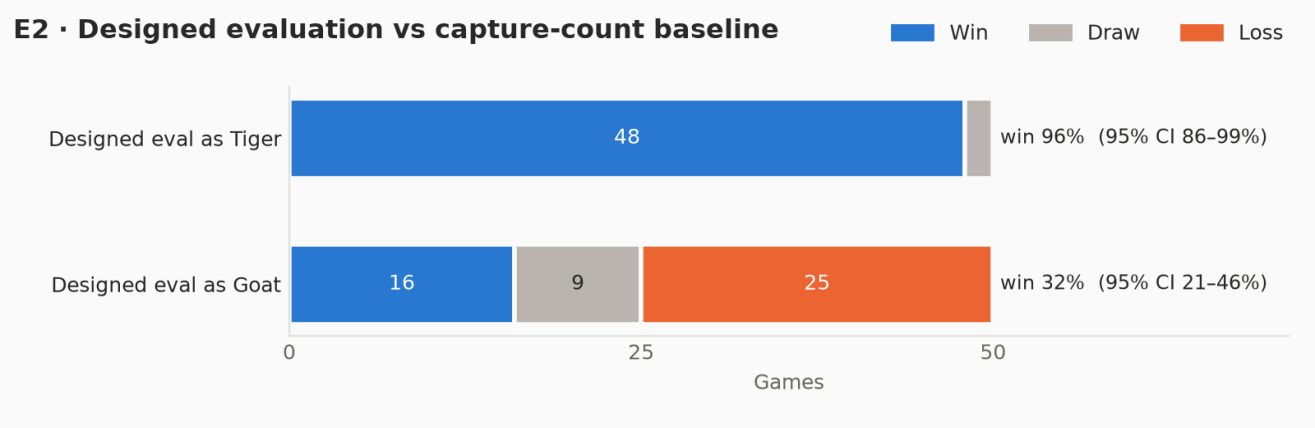
Setup: self-play at depth 3 with alpha-beta. 50 games with the designed evaluation as tiger and 50 as goat, against the capture-count baseline. Game *i* starts after a random 4-ply opening (seed 2026 + *i*), with the same 50 openings for both match-ups.

Table 3. Results for the designed evaluation

Designed eval plays	Games	W	D	L	Win rate (95% Wilson CI)	Mean goats captured
Tiger	50	48	2	0	96% (86–99%)	4.96
Goat	50	16	9	25	32% (21–46%)	3.68

All 11 draws were threefold repetitions; no game reached the 200-ply cap. Mean game length was 45.5 plies (designed as tiger) and 46.2 plies (designed as goat). Outcome breakdown and capture distribution: Appendix Tables A4–A5.

Figure 3. `results/e2_results.png`: win/draw/loss for the designed evaluation as tiger and as goat.



5.3 E3: Alpha-beta's best case and worst case (move ordering)

Setup: the search code is unchanged. `baghchal/ordering.py` wraps the game so that `legal_moves` returns the same moves in a different order, which is the only thing the search sees (§6.2 of the PRD). There are four orders:

- **Worst:** exact child values, worst move for the side to move first.
- **R10:** the fixed rule order, as in E1.
- **Heuristic:** `designed_eval` of each child, best first.
- **Best:** exact child values, best first.

The exact values for best and worst order come from a separate alpha-beta search that is not counted, so these two orders measure the bounds rather than offering a practical agent. E3 runs the same 30 positions at depths 2–4 with the designed evaluation, and also uniform synthetic trees (b = 2, 4, 8) where the textbook formulas are exact. Run with `python -m e3` (about 15 s). Minimax's node count and the R10 alpha-beta count reproduce E1 exactly. Every alpha-beta run returned minimax's root score.

Table 4. Median nodes per move under each ordering (in brackets: median share of minimax's nodes)

Phase	Depth	Minimax	AB worst	AB R10	AB heuristic	AB best
Placement	2	240	240 (100%)	79 (37.8%)	45 (23.5%)	44 (18.7%)
Placement	3	3,055	3,017 (99.2%)	671 (26.2%)	280 (9.8%)	246 (8.4%)
Placement	4	45,746	23,417 (86.1%)	2,627 (8.4%)	720 (2.3%)	650 (1.5%)
Movement	2	68	68 (100%)	46 (66.7%)	25 (36.8%)	25 (36.2%)
Movement	3	615	615 (99.9%)	252 (47.3%)	117 (21.4%)	77 (15.3%)
Movement	4	4,536	4,353 (90.6%)	818 (18.6%)	280 (7.0%)	190 (4.1%)
All	4	6,996	5,678 (86.2%)	1,214 (14.1%)	453 (4.8%)	233 (2.5%)

Depth-4 effective branching factor b*: placement 14.4 (minimax) → 12.1 / 6.9 / 4.9 / 4.8 (AB worst / R10 / heuristic / best), with $\sqrt{b} \approx 3.8$. Movement 7.9 → 7.9 / 5.1 / 3.8 / 3.4, with $\sqrt{b} \approx 2.8$.

Table 5. Uniform trees, b = 8: leaves evaluated

	d = 2	d = 3	d = 4	d = 5
Minimax, b^d	64	512	4,096	32,768
AB worst case (tree built so that each move beats the last)	64	512	4,096	32,768
AB, random leaf values, R10 order	34	279	1,107	4,317
AB best order	15	71	127	575
Knuth–Moore formula b^⌈d/2⌉ + b^⌊d/2⌋ − 1	15	71	127	575

The match is exact for every b and d tested (`results/e3_tables.md`), and the tests in `tests/test_ordering.py` enforce it.

Figure 4. `results/e3_ordering_nodes.png`: median nodes against depth (log scale) for minimax and alpha-beta in worst, R10 and best order.

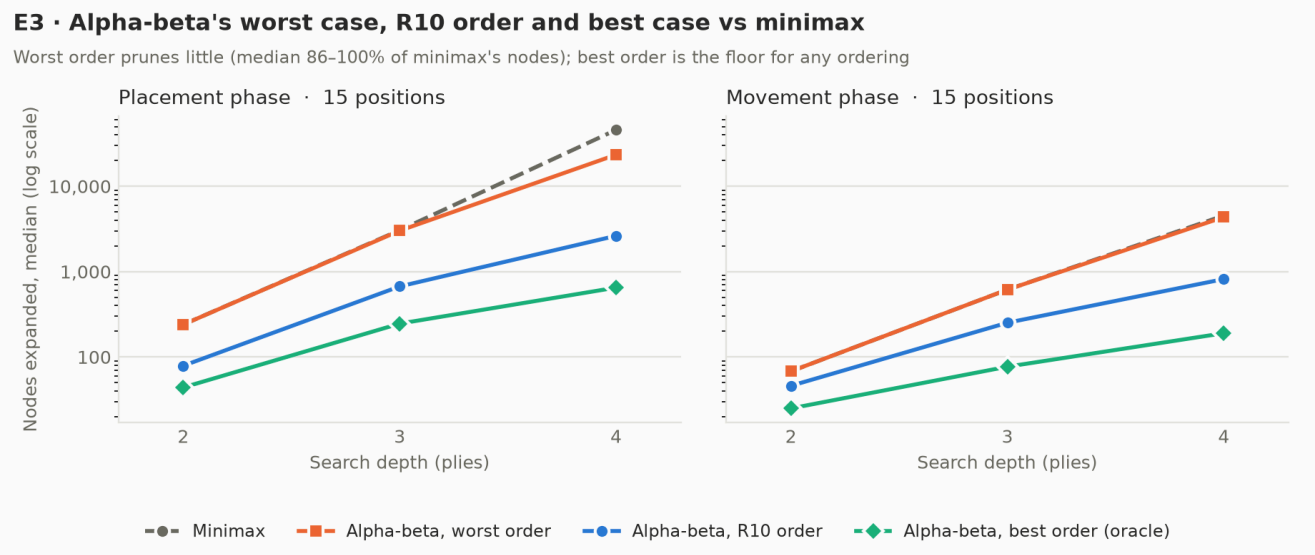


Figure 5. `results/e3_ratio_d4.png`: alpha-beta's nodes as a share of minimax's at depth 4, for each ordering and phase.

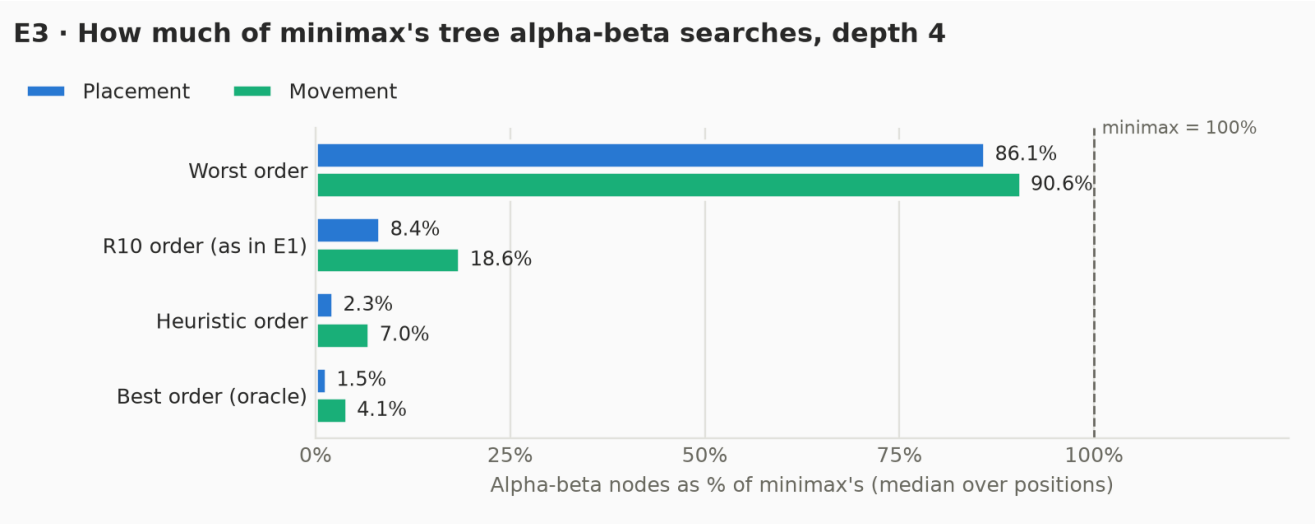
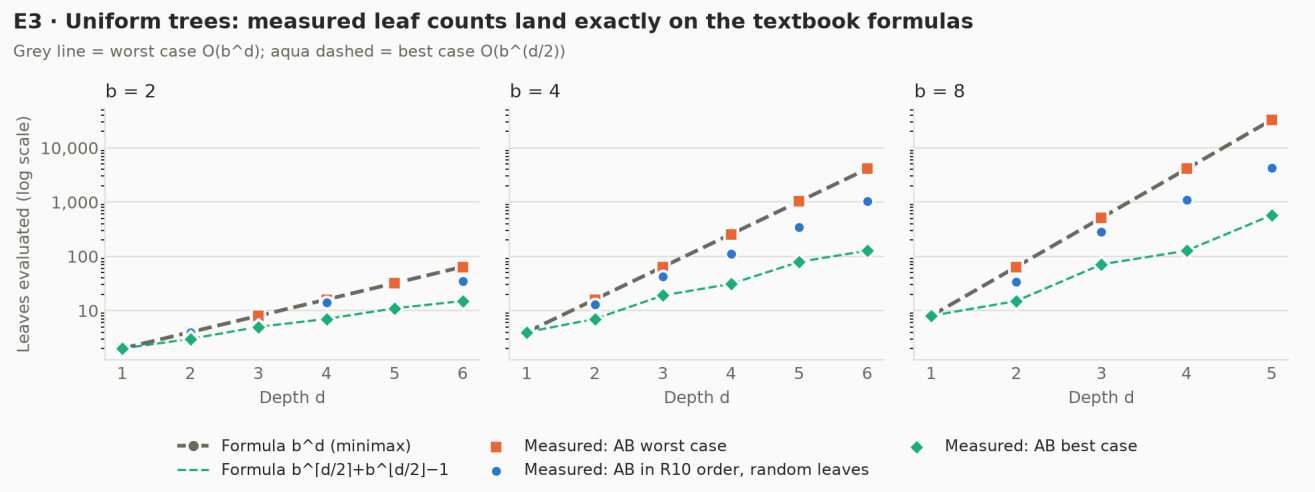


Figure 6. `results/e3_trees.png`: uniform trees, measured leaf counts against the b^d and Knuth–Moore formulas.



6. Analysis

6.1 E1: what alpha-beta saves, and why

What worked. Alpha-beta was correct and much cheaper. It returned minimax's root score on all 90 position/depth pairs. Its share of minimax's nodes fell from 54% at depth 2 to 35% at depth 3 and 14% at depth 4 (Table 1). The saving grows with depth because each cutoff removes a whole subtree. The deeper the search, the larger the subtrees that get cut, and cutoffs at one level compound with cutoffs below. In Figure 1 this shows as a lower slope: on a log scale, alpha-beta's line rises less steeply than minimax's, and the gap widens with depth.

Time follows nodes. The time ratios track the node ratios closely: at depth 4, alpha-beta takes 16% of minimax's time for 14% of its nodes. Both searches cost about 2.5–3.5 µs per node. The bookkeeping alpha-beta adds per node is negligible, and the whole gain comes from expanding fewer nodes. Figure 2 has the same shape as Figure 1.

Placement saves more than movement at every depth (8.4% vs 18.6% at depth 4). The phase changes the branching factor: minimax's b* is about 14–15 in placement and about 8 in movement (Table 2). Once a node is refuted, all of its remaining siblings are skipped, and in placement there are more siblings to skip. A larger b therefore means a larger saving from every cutoff.

What didn't reach the ideal. With perfect move ordering, alpha-beta's b* would be about √b: roughly 3.8 in placement and 2.8 in movement. We measured 6.9 and 5.1 at depth 4. The reason is move order. We search moves in the fixed rule order R10 (ascending point number), which has nothing to do with which move is best. The refuting move is often found late, so many siblings are searched before the cutoff happens. This was a deliberate choice: E1 measures plain alpha-beta. E3 (§5.3, §6.4) confirms that move order is the cause. With the best moves first, the same search drops to b* 4.8 in placement and 3.4 in movement.

6.2 E2: the designed evaluation against the capture-only baseline

As tiger, the designed evaluation is decisive: 48 wins, 2 draws, no losses. All 48 wins came from five captures. The baseline goat sees only captures. It avoids a capture it can see inside its 3-ply horizon, but every non-losing move scores the same (0). The tie rule R11 then picks the first such move in R10 order, the lowest-numbered safe point, so the baseline goat places its goats in a fixed pattern without regard to structure. From the start position, the baseline places at point 2, the first point no tiger can jump to, while the designed evaluation places at point 12, the centre. The designed tiger values `threat` and `mob`, so it steers

toward positions where several goats are attacked at once. A threat on two goats can't be parried with one move, and the baseline can't see the second capture coming until it is inside its horizon, too late to stop it.

As goat, the designed evaluation wins only 32% (16 wins, 9 draws, 25 losses). The goat side is much harder:

- Goats lose a piece to a single oversight and must avoid oversights for the whole game. Tigers only need to find one.
- A depth-3 search from a goat move sees goat–tiger–goat. That catches an immediate jump, but not a tiger move that sets up a double attack two moves later.
- `conn` and `hub` reward a solid wall, but a wall built from goats still in hand is vulnerable while placement is going on.

The designed goat still won 16 games, all by trapping (Table A4). The baseline tiger has no notion of mobility, so it is happy to walk into a corner as long as no capture is visible. It also explains the spread in Table A5: as goat, the designed evaluation lost between 0 and 5 goats, rather than the near-certain 5 the baseline goat lost.

A flat evaluation is cheap but uninformed. Over a whole game, the baseline searched fewer nodes than the designed evaluation (about 3,900 vs 11,700 per game when the designed side was goat). A capture-only evaluation gives most leaves the same value. Equal values make `alpha ≥ beta` happen sooner, so alpha-beta prunes more. The baseline is faster per move but cannot tell good positions from bad ones.

6.3 What the games looked like

Draws were all threefold repetitions (11 of 100 games); no game reached the 200-ply cap, although the longest went to 196 plies. A repetition happens when neither side can make progress within its horizon. The tigers can't find a capture, the goats can't close the trap, and both sides shuffle back and forth between equally scored positions. The fixed tie-breaking rule makes this more likely: in a position where nothing changes the score, a side picks the same move every time.

Games were short and were usually decided during placement. The placement phase lasts 39 plies (20 goat placements and 19 tiger replies). The median game was 40 plies with the designed evaluation as tiger and 36.5 as goat, and 55 of the 100 games were over within 40 plies. Goats are most vulnerable while placing, because each new goat is a new target and a solid wall doesn't exist yet. This is also the phase where alpha-beta saves the most (§6.1), which is fortunate: it is where the extra depth matters.

6.4 E3: best case, worst case, and which algorithm wins when

Minimax has no best or worst case. Alpha-beta's depends entirely on move order. Minimax expands every node whatever the order: its count was identical under all four orders, and the tests check this. Alpha-beta's count at depth 4 ranged from 86% of minimax's nodes in worst order to 2.5% in best order (Table 4), on the same positions and with the same code. The reason is the cutoff test `alpha ≥ beta`. It fires as soon as one side has seen a move good enough to make the other side avoid this line. In best order, the very first move sets that bound, and each opponent alternative is refuted after one reply. In worst order, every new move is better than the previous one, so the bound never tightens in time to cut anything.

The uniform trees confirm the textbook exactly. - In best order, alpha-beta evaluated exactly $b^{\lceil d/2 \rceil} + b^{\lfloor d/2 \rfloor} - 1$ leaves for every b and d tested (Table 5). That is about $2 \cdot b^{(d/2)}$, so the effective branching factor becomes $\sqrt{b}$, and the same work searches about twice as deep. - On a tree built so that each move is better than the one before it, alpha-beta evaluated all b^d leaves, the same as minimax. - Random leaf values in a fixed order fall in between.

On Bagh-Chal, simply reversing the oracle order is not quite the worst possible order. It equals minimax on 23 of the 30 positions at depth 2, but at depth 4 a few cutoffs still fire from bounds set by an ancestor's earlier

siblings, so it searches between 37% and 100% of minimax's nodes. This is still effectively no pruning. Placement saves more than movement under every order, for the reason given in §6.1.

A cheap ordering gets most of the way. Sorting moves by the static evaluation of the child brings depth 4 to 4.8% of minimax's nodes overall (placement 2.3%), against 2.5% for the oracle and 14.1% for R10 order. At depth 4 this does not yet save time. Calling `designed_eval` on every child to sort them costs more than the nodes saved (placement: 12.0 ms vs 7.3 ms median for R10 order). Cheaper orderings, such as captures first, or deeper searches would change that.

Which algorithm does better, and when. - **Alpha-beta never does worse in nodes, and it returns the same value.** - **Alpha-beta wins most** with deep searches (54% → 35% → 14% of minimax's nodes from depth 2 to 4), with many moves per position (placement saves twice as much as movement), and with good move ordering. - **They tie** at depth 1, where nothing can be cut, and in the worst order, where alpha-beta does minimax's work plus a few comparisons. - **Minimax is the better choice** only when the exact value of *every* root move is needed, for example to rank or explain all moves, since alpha-beta gives only a bound for moves it prunes, or when the simplest code matters, as for a reference implementation. We use minimax as the reference that alpha-beta is tested against.

Full comparison: `docs/algorithm_comparison.md`.

6.5 Conclusions

1. **(a)** Alpha-beta is exact and its advantage grows with depth: at depth 4 it needs 14% of minimax's nodes overall and 8.4% in placement, with time saved in proportion. Without move ordering, it remains well short of the √b best case.
2. **(b)** The designed evaluation is clearly better than counting captures. The difference is overwhelming for the tiger and real but smaller for the goat, whose defensive task needs more depth than 3 plies.
3. **(c)** Alpha-beta's cost is set by move order. It ranges from about minimax's cost in the worst case (exactly b^d on a constructed tree) to exactly b^⌈d/2⌉ + b^⌊d/2⌋ − 1 leaves in the best case. A heuristic order reaches 4.8% of minimax's nodes at depth 4. Alpha-beta is never worse than minimax, and minimax is preferable only when every move's exact value is needed.

7. Limitations and what we'd do differently

Limitations

- **Shallow search.** Depth 3–4 cannot see the multi-move plans that decide many goat games (§6.2).
- **No search enhancements in the agent.** The agent plays with R10 order and no transposition table, iterative deepening or quiescence search; these were deliberate non-goals (PRD §2). As a result, alpha-beta's b* stays well above √b, and tactical positions can be cut off in the middle of a capture sequence (the horizon effect). E3 measures move ordering but does not switch the agent to it.
- **E3's bounds are empirical on Bagh-Chal.** The best and worst orders use an oracle, so they bound what ordering can achieve rather than describing a playable agent. On Bagh-Chal's irregular trees, the reversed oracle order is close to, but not exactly, the worst possible order (§6.4).
- **Weights not tuned.** The weights were chosen by reasoning, not by experiment. We don't know how close to good they are.
- **E2 is limited.** It plays against one weak opponent at one depth (3), with 50 games per side. The confidence intervals are wide (21–46% for the goat win rate). The result shows the designed evaluation beats counting captures, not that it plays well against strong opponents.
- **Test positions are not a uniform sample.** About 70% of the moves that produced the E1 positions were chosen by the designed evaluation itself, so the suite leans toward positions it tends to reach. Every position also starts with an empty repetition history.

- **One rule set.** Results are for our frozen rules R1–R11; variants with compulsory capture or different repetition rules may behave differently.

What we would do differently

1. **Order moves before searching.** E3 shows that a heuristic order cuts depth-4 nodes from 14.1% to 4.8% of minimax's. A cheaper key (captures first, then moves that create threats) should keep that saving without the cost of evaluating every child, so it would also save time.
2. **Tune the weights** with a short round-robin over 3–4 candidate weight sets (PRD §11.2), especially to strengthen the goat side.
3. **Use iterative deepening with a time limit,** so the agent searches as deep as the clock allows rather than to a fixed depth. The goat side would gain most.
4. **Run E2 at several depths,** and also against the designed evaluation itself, to see whether the tiger/goat asymmetry shrinks with depth.

8. References and declaration of AI tool use

References 1. S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed., Pearson, 2020, Ch. 5 (Adversarial Search). 2. BCSE306L Artificial Intelligence, Module 3: Adversarial Search, course material, VIT Vellore. **[CHECK: exact title of the course notes.]** 3. **[CHECK: the source the team used for the Bagh-Chal rules, if any. If none, delete this line; the rules as played are stated in full in §2.1.]**

Declaration of external AI tool use

We used Claude (Anthropic) during scoping and to help write the Review 1 proposal. Our Review 1 declaration said that any further use would be declared here, and it is:

- **Code.** Claude Code (Anthropic's coding agent) wrote the game rules (`board.py`, `state.py`, `rules.py`, `game.py`), the tests, the adapter, the command-line interface, the benchmarks and the experiment harness.
- **Search and evaluation.** On 2026-10-13, at Pranav's direction while Shalom and Vishal were unavailable, Claude Code also wrote `search.py` (minimax, alpha-beta and move choice) and `evaluation.py` (the six features, both evaluation functions and the weights). This supersedes our Review 1 statement that Claude wrote no part of the search or evaluation. Shalom is responsible for the search and Vishal for the evaluation. **[CHECK: add whether either of them reviewed or changed the code, and when.]**
- **Report.** On 2026-10-22, at Pranav's direction, Claude Code wrote the text of this report, including the analysis and conclusions (§6), the implementation notes (§4), the tables and the slide text.
- **Experiment E3 and the comparison.** Later on 2026-10-22, at Pranav's direction and to meet the professor's request for a best-case/worst-case comparison, Claude Code wrote the move-ordering wrapper (`baghchal/ordering.py`), the E3 experiment (`e3/`) and its tests. It also wrote §2.3, §5.3 and §6.4 of this report, `docs/algorithm_comparison.md` and `docs/ai_ml_usage.md`. `search.py` and `evaluation.py` were not changed.
- **What we did.** Pranav directed the work, chose the scope and the frozen rules recorded in the project specification, and wrote the experiment framework (`experiments/`). **[CHECK: add each member's own contributions; Claude helped with the specification during scoping, as declared above.]** Any further changes by team members are recorded in the project specification (`docs/PRD.md` §10) and the git history.

All three of us have checked the results against the code and output, and each of us can explain the search, the evaluation and the experiments. **[CHECK: every member must confirm this sentence is true before submission.]**

Appendix

E3 tables (all depths, b* and uniform trees): `results/e3_tables.md`; the full minimax vs alpha-beta comparison: `docs/algorithm_comparison.md`.

Per-position E1 tables, root agreement and E2 outcome breakdowns: `docs/report/tables_appendix.md` (regenerate with `python tools/report_tables.py`).

Appendix A. Tables

Generated by `python tools/report_tables.py` from `results/e1_raw.csv` and `results/e2_raw.csv`. Tables only; interpretation is written by the team.

Table A1. E1 per position, depth 4

Position	Phase	To move	Minimax nodes	Alpha-beta nodes	AB / MM	Root score	Minimax move	Alpha-beta move
P01	placement	GOAT	97,041	2,627	2.7%	43	G@0	G@0
P02	placement	TIGER	74,215	9,195	12.4%	130	T 0-5	T 0-5
P03	placement	TIGER	46,607	6,223	13.4%	213	T 15x10>5	T 15x10>5
P04	placement	GOAT	58,269	4,525	7.8%	228	G@9	G@9
P05	placement	TIGER	66,947	4,804	7.2%	109	T 8-7	T 8-7
P06	placement	GOAT	46,325	3,876	8.4%	128	G@9	G@9
P07	placement	GOAT	45,746	17,079	37.3%	109	G@15	G@15
P08	placement	TIGER	37,109	2,086	5.6%	-8	T 12-7	T 12-7
P09	placement	GOAT	29,107	2,110	7.2%	999996	G@0	G@0
P10	placement	TIGER	48,085	2,353	4.9%	999999	T 21x16>11	T 21x16>11
P11	placement	GOAT	22,947	4,203	18.3%	392	G@15	G@15
P12	placement	GOAT	3,335	436	13.1%	-51	G@1	G@1
P13	placement	TIGER	9,255	770	8.3%	138	T 8-3	T 8-3
P14	placement	TIGER	4,697	1,222	26.0%	143	T 16-20	T 16-20
P15	placement	GOAT	1,444	286	19.8%	139	G@6	G@6
M01	movement	GOAT	2,014	267	13.3%	91	G 5-0	G 5-0
M02	movement	GOAT	7,160	931	13.0%	356	G 1-0	G 1-0
M03	movement	GOAT	2,396	629	26.3%	216	G 18-24	G 18-24
M04	movement	TIGER	2,193	601	27.4%	228	T 18-12	T 18-12
M05	movement	TIGER	6,324	1,625	25.7%	999999	T 19x18>17	T 19x18>17
M06	movement	GOAT	4,536	845	18.6%	225	G 10-5	G 10-5
M07	movement	GOAT	5,589	1,035	18.5%	213	G 22-16	G 22-16
M08	movement	TIGER	293	104	35.5%	-91	T 15-10	T 15-10
M09	movement	TIGER	3,136	351	11.2%	276	T 7-12	T 7-12
M10	movement	TIGER	3,707	818	22.1%	314	T 13-18	T 13-18
M11	movement	TIGER	3,951	1,206	30.5%	196	T 16-11	T 16-11
M12	movement	GOAT	4,976	735	14.8%	291	G 19-24	G 19-24
M13	movement	TIGER	8,845	1,474	16.7%	349	T 13-18	T 13-18
M14	movement	GOAT	6,831	1,327	19.4%	352	G 18-22	G 18-22
M15	movement	GOAT	11,246	752	6.7%	349	G 5-10	G 5-10

Table A2. E1 per position, depth 3

Position	Phase	To move	Minimax nodes	Alpha-beta nodes	AB / MM	Root score	Minimax move	Alpha-beta move
P01	placement	GOAT	5,802	604	10.4%	29	G@0	G@0
P02	placement	TIGER	4,399	1,606	36.5%	147	T 0-6	T 0-6
P03	placement	TIGER	3,051	825	27.0%	243	T 11x12>13	T 11x12>13
P04	placement	GOAT	3,988	1,047	26.3%	219	G@9	G@9
P05	placement	TIGER	4,737	840	17.7%	133	T 8-7	T 8-7
P06	placement	GOAT	3,486	754	21.6%	119	G@9	G@9
P07	placement	GOAT	3,364	1,658	49.3%	100	G@15	G@15
P08	placement	TIGER	3,055	526	17.2%	11	T 3-8	T 3-8
P09	placement	GOAT	2,661	611	23.0%	403	G@2	G@2
P10	placement	TIGER	3,535	671	19.0%	999999	T 21x16>11	T 21x16>11
P11	placement	GOAT	1,974	725	36.7%	290	G@15	G@15
P12	placement	GOAT	470	132	28.1%	-54	G@1	G@1
P13	placement	TIGER	1,132	290	25.6%	150	T 18-24	T 18-24
P14	placement	TIGER	743	364	49.0%	158	T 17-18	T 17-18
P15	placement	GOAT	234	104	44.4%	93	G@6	G@6
M01	movement	GOAT	352	121	34.4%	32	G 5-0	G 5-0
M02	movement	GOAT	922	486	52.7%	341	G 14-13	G 14-13
M03	movement	GOAT	615	326	53.0%	193	G 18-24	G 18-24
M04	movement	TIGER	175	93	53.1%	234	T 18-12	T 18-12
M05	movement	TIGER	565	282	49.9%	999999	T 19x18>17	T 19x18>17
M06	movement	GOAT	756	232	30.7%	185	G 10-5	G 10-5
M07	movement	GOAT	921	264	28.7%	156	G 21-16	G 21-16
M08	movement	TIGER	50	30	60.0%	-28	T 15-10	T 15-10
M09	movement	TIGER	366	173	47.3%	287	T 16-11	T 16-11
M10	movement	TIGER	387	252	65.1%	334	T 24-23	T 24-23
M11	movement	TIGER	358	197	55.0%	227	T 12-11	T 12-11
M12	movement	GOAT	648	204	31.5%	195	G 19-24	G 19-24
M13	movement	TIGER	817	337	41.2%	352	T 11-16	T 11-16
M14	movement	GOAT	988	434	43.9%	338	G 20-21	G 20-21
M15	movement	GOAT	1,415	262	18.5%	342	G 6-10	G 6-10

Table A3. E1 root agreement between minimax and alpha-beta

Depth	Pairs	Same score	Same move	Different move, same score
2	30	30	30	0
3	30	30	30	0
4	30	30	30	0

Table A4. E2 outcomes by how the game ended

Designed eval plays	Tigers won	of which by 5 captures	Goats won	Draw (repetition)	Draw (move cap)	Mean plies	Median plies	Mean goats captured
Tiger	48	48	0	2	0	45.5	40	4.96
Goat	25	25	16	9	0	46.2	36.5	3.68

Goat wins are always traps (R8.2). A tiger win not by 5 captures would be R8.3 (goat has no move).

Table A5. E2 goats captured per game

Designed eval plays	0	1	2	3	4	5
Tiger	0	0	0	0	2	48
Goat	1	7	5	6	6	25

Appendix B. Implementation notes

> **Draft status.** Drafted by Claude Code from the code on 2026-10-13. It describes what the code does. The five places that were marked **[OWNER: …]** now hold placeholder text written by Claude Code on 2026-10-22 and tagged **[DRAFT …]**. Each owner must check it and rewrite it in their own words, then delete the tag. Treat any tag still present as unfinished. Per PRD §10, Claude Code wrote `search.py` and `evaluation.py` at Pranav's direction. Shalom owns the search and Vishal owns the evaluation, and the report's declaration (§8) must say so.

4.1 State representation

A position is an immutable `State` (a frozen dataclass, `baghchal/state.py`). The board is a 25-tuple indexed by `i = 5·row + col`, where each entry is `EMPTY = 0`, `GOAT = 1` or `TIGER = 2`. Alongside the board the state holds the side to move, goats in hand `h`, goats captured `k` and the ply count `n`. These five fields define equality and hashing, matching the formal state `⟨B, h, k, p, H, n⟩` of §2.

Four more fields are bookkeeping and take no part in equality:

- `tigers`: the sorted points of the four tigers. Tiger moves can then be generated without scanning the board.
- `history` and `reps`: the repetition record for R8.4 (see §4.4).
- `moves_cache`: the legal moves, filled the first time they are asked for.

`apply(state, move)` never changes its argument. It copies the board into a list, edits two or three points, and builds a new `State`. Immutability means a search can hold any number of states without one move corrupting another. Every test checks it: across 200 random games, the input of `apply` is compared before and after every move.

Text boards (`"T...T/...../...../...../T...T"`) are parsed and validated only at the program's edges, by the CLI and when loading positions. The checks are: 4 tigers, goats on board + in hand + captured = 20, and legal characters and row lengths. Inside the search no state is ever re-validated.

[DRAFT: placeholder written by Claude Code on 2026-10-22. Shalom/Vishal: check it, rewrite it in your own words, then delete this tag.] Minimax explores many alternative futures from the same position, so each child must be a separate position that can't disturb its parent or siblings. With an immutable state, `apply` makes a fresh child and the search never has to undo a move. A tuple is also hashable, so the same value can serve as a repetition key (R8.4) and be compared cheaply.

4.2 Board topology generated from the rule

Adjacency is never typed in by hand. `baghchal/board.py` generates it at import time from rule R1. Two points are adjacent when they differ by at most 1 in each coordinate, and a diagonal step is allowed only from a point where `row + col` is even. From that one rule the module builds:

Table	Contents	Check in the tests
`NEIGHBOURS[x]`	neighbours of `x`, ascending	adjacency is symmetric; no diagonal leaves an odd point
`DEGREE[x]`	number of neighbours	degrees 3 (12 points), 4 (4), 5 (4), 8 (5: points 6, 8, 12, 16, 18)
`EDGES`	each edge once, `(a, b)` with `a < b`	56 edges
`JUMPS`	every `(x, over, land)` triple	80 directed jumps
`TIGER_TARGETS[x]`	every step and jump from `x`, sorted by landing point	equals `NEIGHBOURS` plus `JUMPS`

The tables are generated rather than typed because a single wrong entry in a hand-typed 25-point adjacency list would quietly change the game, and with it every experiment. Generating from the rule leaves one line to get right, and the known counts above (56 edges, 13 even points, 80 jumps) test that line.

4.3 The jump lemma

R2 states that if `over` is a neighbour of `x` and `land = 2·over − x` is on the board, then `land` is a neighbour of `over` along the same line. The jump table is therefore built from coordinates alone. For each neighbour `over` of `x`, it continues one more step in the same direction and keeps the triple if the landing point is on the board.

The lemma holds because diagonals exist only on even points. Moving one diagonal step changes `row + col` by 0 or ±2, so parity is preserved: if `x` is even, `over` and `land` are even too, and the diagonal continues. The tests check the lemma on all 80 triples instead of assuming it. During play a jump is therefore just two lookups: `over` must hold a goat and `land` must be empty.

4.4 Move generation, move order and repetition

`baghchal/rules.py` implements R5–R8.

- **Goat placement (R5):** one move per empty point, ascending.
- **Goat steps (R6):** for each goat in index order, each empty neighbour in ascending order.
- **Tiger moves (R7):** for each tiger in `tigers` order, the precomputed `TIGER_TARGETS` list. A step needs an empty landing point; a jump also needs a goat on `over`. Because that list is sorted by landing point, steps and jumps come out interleaved in `(from, to)` order. This is exactly R10, and no sorting happens at runtime.

Moves are single objects created once at import, such as `PLACE[12]` and `JUMP[0][6]`, so generating moves creates no new objects. They print as `G@12`, `G 7–12`, `T 0–1` and `T 0x6>12`.

The legal moves are stored on the state the first time they are computed. That avoids repeated work, because the terminal test (R8.2/R8.3: "does the side to move have a move?") and the search's own move loop both need the same list.

Repetition (R8.4) is tracked with a linked chain of positions, `(key, previous)`, where `key = (board, side to move, h)`. The chain holds only the positions since the last placement or capture, because those moves are irreversible:

- a placement lowers `h`, and nothing ever raises it;

- a capture permanently removes a goat.

So no position from before an irreversible move can recur, and the short chain is exactly as good as the full game history. `apply` computes `reps`, the number of times the new position has occurred, once. It walks the chain comparing only every second entry (same side to move), and the terminal test then reads `reps` in constant time.

Terminal test (R8) checks the conditions in the stated order: `k ≥ 5`; tiger to move with no move; goat to move with no move; three repetitions; the 200-ply cap. During placement the goat always has a move: the number of empty points is `1 + h + k ≥ 2`, so that check is skipped.

The rules were checked by *perft*, counting the positions reachable in exactly *d* plies from the start. The counts were 21, 252, 5,052 and 68,204 for *d* = 1–4, matching two independent implementations. The rules code handles about 1 million `legal_moves` or `apply` calls per second on the team's laptop under Python 3.14 (about 0.6 million under Python 3.9).

4.5 Search and node counting

`baghchal/search.py` contains `minimax`, `alphabeta` and `choose_move`. The search sees the game only through five methods (`legal_moves`, `apply`, `is_terminal`, `utility`, `is_max_to_move`, §6.2 of the PRD). It does not import the rules module, so the same code is tested on the Russell & Norvig Fig. 5.2 tree and then runs unchanged on Bagh-Chal.

Each recursive call does four things in order:

1. **Count the node.** `stats.nodes += 1` is the first statement of every call: root, cutoff and terminal calls included.
2. **Terminal?** Return the exact utility `±(1,000,000 − ply)` or 0 (R9). This happens even when the depth has also run out, because a known result beats an estimate.
3. **Depth 0?** Return the heuristic `evaluate(state)`.
4. **Otherwise,** search every child in R10 order and keep the best value for the side to move. Tigers maximise and goats minimise (R9).

Counting at the very top means minimax and alpha-beta are measured the same way, which is what makes the E1 comparison fair. It also gives two checkable facts:

- a depth-1 minimax search expands exactly `1 + (number of legal moves)` nodes;
- on the textbook tree, minimax expands 13 nodes and alpha-beta 11 (the second MIN node is cut after its first leaf).

Both are tests, and `python -m experiments check-engine` checks the first again on real positions.

Search depth versus distance from the root: `depth` is the number of plies still to search, and it counts down. `ply` is the distance from the root, and it counts up. `ply` feeds the R9 utility, so a win found sooner scores higher and the agent prefers faster wins and slower losses.

Alpha-beta passes the bounds `alpha` (the best MAX can already guarantee on this path) and `beta` (the best for MIN) down the tree, and stops searching a node's children once `alpha ≥ beta`. The root call uses `(−∞, +∞)`.

[DRAFT: placeholder written by Claude Code on 2026-10-22. Shalom: check it, rewrite it in your own words, then delete this tag.] In Fig. 5.2 the root A is MAX and B, C, D are MIN nodes. B's leaves 3, 12, 8 give B = 3, so after B the root is guaranteed at least 3 and alpha becomes 3. C's first leaf is 2. Because C is a MIN node, its value can only be 2 or lower, whatever its other two leaves are. MAX already has 3, so it will never choose C, and the remaining leaves (4 and 6) can't affect the root: alpha (3) ≥ C's bound (2), so C is cut. D is

searched in full (14, 5, 2 → 2), and the root is max(3, ≤2, 2) = 3, the same as minimax. Only those two leaves are skipped: 11 nodes instead of 13.

4.6 Tie-breaking

At every node the search keeps the **first** move that reaches the best value. It uses strict `>` for MAX and strict `<` for MIN (R11), and children are visited in the fixed R10 order, so results are reproducible. Node counts were identical across repeated runs in every test and in all of E1, which aborts if they differ.

Alpha-beta is guaranteed to return the same *value* as minimax at the root, but not always the same *move*. When moves tie, a pruned subtree can return a bound equal to the best value without being searched in full, and the strict comparison then keeps an earlier move. E1 therefore compares root scores, not moves. They matched on all 90 position/depth pairs.

[DRAFT: placeholder written by Claude Code on 2026-10-22. Shalom: check it, rewrite it in your own words, then delete this tag.] Example (`test_r11_first_best_move_kept_on_ties`): the root is MAX with three MIN children whose leaves are [5, 7], [6, 5] and [5, 9], so every child is worth 5. Minimax sees child 0 = 5 first. Children 1 and 2 also score 5, but 5 > 5 is false, so it keeps move 0. Alpha-beta gives the same answer: after child 0, alpha = 5. Child 2's first leaf is 5 ≤ alpha, so it is cut before the 9 is seen; its bound of 5 doesn't beat 5 either. Both return move 0 with score 5. If the comparison were `≥`, the last tied move would win instead, and minimax and alpha-beta could pick different moves, because a pruned child can return a bound equal to the best value.

4.7 Evaluation function

`baghchal/evaluation.py` computes the six features of §6.4 of the PRD in one pass, whoever is to move:

- **mob, threat, trapped:** for each tiger, walk its `TIGER_TARGETS`. Each legal step adds to `mob`. Each legal jump marks the jumped goat in a bitmask, so a goat attackable by two tigers counts once in `threat`. A tiger with neither is `trapped`.
- **conn:** count the 56 edges with a goat at both ends.
- **hub:** add `DEGREE[x]` for every goat on an even point.
- **capt:** equals `k`.

The evaluation is a single tiger-side score:

```
EVAL = 100·capt + 3·mob + 15·threat − 40·trapped − 2·conn − 1·hub
```

Its magnitude is at most 896 for tigers and 344 for goats, far inside the ±100,000 bound. The bound is also tested on 500 random reachable states. A call takes about 2 µs. The baseline evaluator for E2 is `capt` alone.

[DRAFT: placeholder written by Claude Code on 2026-10-22. Vishal: check it, rewrite it in your own words, then delete this tag.] The initial weights were kept. `capt` (100) is the largest because a capture is permanent and five end the game, so one capture must outweigh any realistic positional swing. `trapped` (40) is next: four trapped tigers win for the goats, but a tiger can be freed, so it counts for less than a capture. `threat` (15) is about a seventh of a capture: an attackable goat is often lost, but it can still be defended. `mob` (3), `conn` (2) and `hub` (1) are small positional terms. They have many units each (mobility up to about 30, 56 edges, hub up to 72), and `hub` is smallest because it overlaps with `conn`. The score is a single number: tigers maximise it and goats minimise it, so one player's gain is exactly the other's loss. That is the zero-sum assumption that minimax and alpha-beta rely on. If each side used its own evaluation, the bounds alpha and beta would refer to different scales and pruning could discard the true best line.

4.8 How the test positions were generated

The 30 E1 positions (`data/positions.json`) come from real play, not hand-typing, so every one is reachable and legal. `python -m experiments generate --seed 2026`:

1. plays 200 games from the start position;
2. chooses each move epsilon-greedily: a uniformly random legal move 30% of the time, otherwise the engine's own depth-2 alpha-beta move with the designed evaluation;
3. collects every non-terminal position after the first 6 plies into a pool (8,792 distinct candidates);
4. draws 15 placement-phase and 15 movement-phase positions from the pool, with the side to move split 8/7 within each phase.

Pure random play isn't used because random goats are captured long before placement ends, so the movement phase would rarely appear. One seed reproduces the whole file; the seed and parameters are stored in its metadata, and the file is committed and never regenerated.

Each position starts with an empty repetition history and ply 0. This is a known simplification.

[DRAFT: placeholder written by Claude Code on 2026-10-22. Pranav: check it, rewrite it in your own words, then delete this tag.] Caveat: 70% of the moves that produced the positions were chosen by the designed evaluation itself, so the suite leans toward positions that evaluation tends to reach. This doesn't bias E1, where both algorithms search the same positions and must return the same value. It does mean the positions aren't a uniform sample of all reachable states. The empty repetition history also means no test position is close to a repetition draw.